QueryAPIGate HOW-TO GUIDE 18 · ACCESS

Restrict a key to specific source IPs

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=18-restrict-a-key-to-ips

Time: 5 minutes. **You'll end up with:** a key that only works from the addresses you list - so if it leaks, it's useless from anywhere else.

This suits a key handed to a partner or service with stable infrastructure: a fixed office egress address, a cloud NAT gateway, a data centre range. It's a second lock, not a replacement for keeping the key secret.

Step 1: Set `allowed_ips`

A list of IPv4 or IPv6 addresses and CIDR ranges, mixed freely:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "office-only", "connections": ["shop"], "allowed_ips": ["203.0.113.5", "198.51.100.0/24"]
}'
```

Each entry is checked when it's saved - `"10.0.0.300"` is refused with `400`: *'10.0.0.300' is not a valid IP address or CIDR range*.

Step 2: See it refuse other addresses

Called from `127.0.0.1`, which isn't on the list:

```shell
curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: <office-only secret>'
# {"error": "Unauthorized", "code": "unauthorized", ...}
```

A refused address looks exactly like a wrong key - `401 Unauthorized`, no hint that the key itself is valid. Someone trying a stolen key learns nothing from the response. The flip side: when a legitimate caller reports `401`, check their address against the list before suspecting the key.

Add the address and it works from the next request - no new secret:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/api-keys/office-only -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"allowed_ips": ["127.0.0.0/8", "::1", "198.51.100.0/24"]}'

curl 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: <office-only secret>'
# [{"id": 1, "total": 42.5, "status": "paid"}, ...]
```

To remove the restriction, set it to `null`. An empty list (`[]`) allows **no** address - the key stops working everywhere, the same as an empty `allowed_tables`.

Which address is "the caller's"?

The address the connection came from - unless you tell QueryAPIGate there's a proxy in front of it. Verified:

- **No proxy setting (the default):** an `X-Forwarded-For` header is ignored. Sending `X-Forwarded-For: 203.0.113.5` from an unlisted address still gets `401` - a caller can't claim an allowed address.
- **`QUERYAPIGATE_TRUST_PROXY=1`:** the address comes from `X-Forwarded-For`, as set by your proxy. `X-Forwarded-For: 192.0.2.44` gets `401`; `X-Forwarded-For: 127.0.0.9` (inside `127.0.0.0/8`) is let in.

So:

- **Behind a proxy or load balancer, set `QUERYAPIGATE_TRUST_PROXY`** to the number of proxies. Without it, every request appears to come from the proxy - list the proxy's address and you've allowed everyone; don't, and you've allowed no one.
- **Without a proxy, leave it unset.** With it set and nothing in front, anyone can write their own `X-Forwarded-For` and pick an allowed address.

The same address is what the server-wide rate limit counts by. See Put QueryAPIGate behind a reverse proxy.

Good to know

- **The admin key is never restricted** by any key's `allowed_ips`. It has no `allowed_ips` of its own - protect it by not handing it out.
- **Roles carry it too.** Keys created from a role get a copy; signed-in (JWT) users are checked against their role's list on every request.
- **Every surface is covered** - saved queries, ad-hoc SQL, `/catalog`, `/openapi.json`'s saved-query section, live events and MCP all check it.
- **The MCP server (`queryapigate mcp`) doesn't read `QUERYAPIGATE_TRUST_PROXY`** - it always uses the address of the connection it received. Put a proxy in front of it and every MCP caller appears to come from the proxy, so an IP-restricted key can only work there if you list the proxy's address. Prefer giving MCP clients keys without `allowed_ips`, or reach the MCP port directly.

Next steps

- Rate-limit a key.
- Give a key an expiry date.
- Give an external partner access to exactly one query.

READ NEXT

Give a key an expiry date

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=18-restrict-a-key-to-ips

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate